



河南大學
HENAN UNIVERSITY



Web编程基础

(2020-2021-2)

任课教师: 丁爽



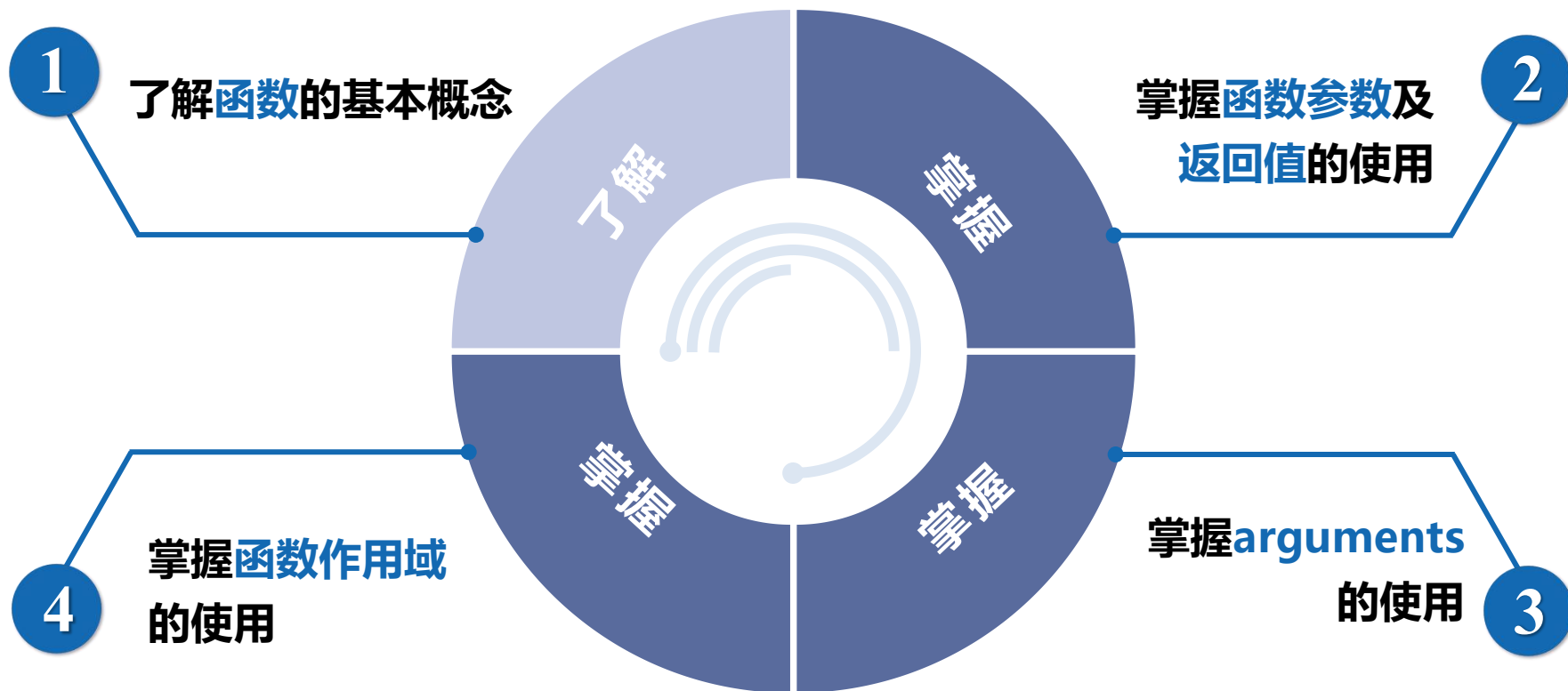
内容回顾

- **JS**实现业务逻辑和页面控制，**JS**引擎逐行执行
- 包括**ECMAScript**、**DOM**、**BOM**
- 三种引入方式，输入输出语句
- 变量的使用
- 弱类型语言，类型转换
- 流程控制：顺序、分支、循环



第4章 JavaScript函数

学习目标



目录



4.1

初识函数

👉 [点击查看本节相关知识点](#)

4.2

函数案例

👉 [点击查看本节相关知识点](#)

4.3

函数进阶

👉 [点击查看本节相关知识点](#)

4.4

作用域

👉 [点击查看本节相关知识点](#)

目录



闭包函数

👉 [点击查看本节相关知识点](#)



预解析

👉 [点击查看本节相关知识点](#)



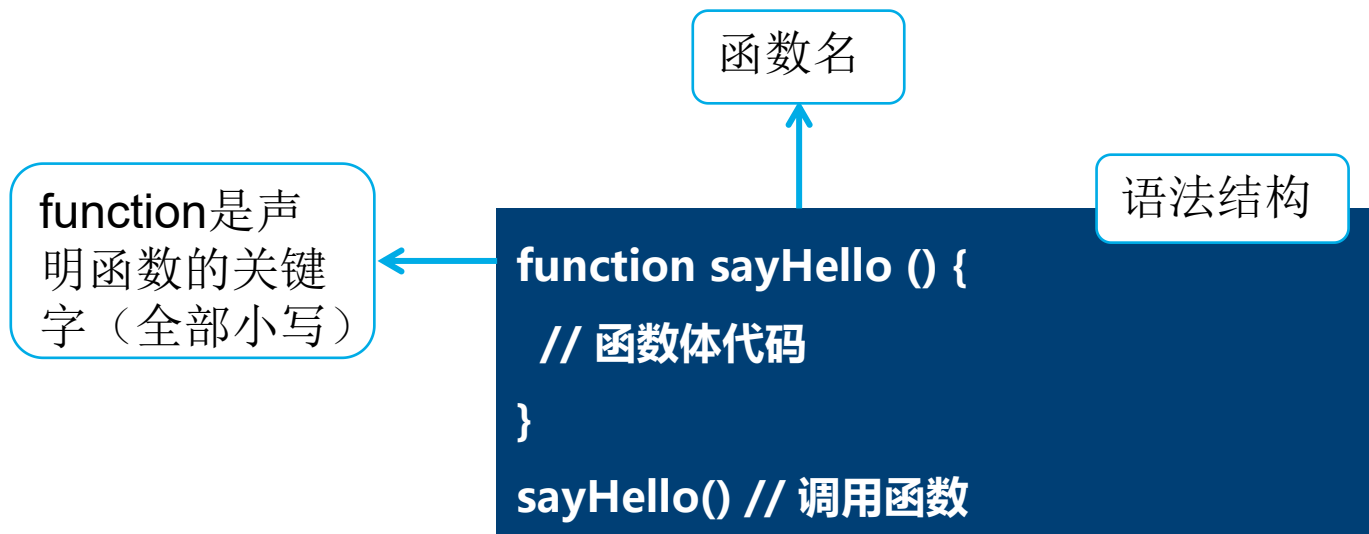
4.1 初识函数

1

函数的使用

避免相同功能代码的重复编写，将程序中的代码模块化，提高程序的可读性，减少开发者的工作量，便于后期的维护

函数在使用时分为两步，声明函数和调用函数





4.1 初识函数

2

什么是函数

在编写代码时，可能会出现非常多的相同代码，或者功能类似的代码，这些代码可能需要大量重复使用，此时就可以使用JavaScript中的[函数](#)。

函数：求n~m的累加和

定义getSum()

```
function getSum(num1, num2) {  
    var sum = 0;  
    for (var i = num1; i <= num2; i++) { sum += i; }  
    console.log(sum); // 函数执行结束后，将结果输出  
}
```

调用getSum()

```
getSum(1, 100); // 输出结果：5050  
getSum(10, 50); // 输出结果：1230
```




4.1 初识函数

3

函数的参数

函数的参数分为形参和实参：

- **形参**：在声明函数时，在函数名称后面的小括号中添加的一些参数
- **实参**：当函数调用的时候，需要传递相应的参数，这些参数称为实参

语法结构

```
function 函数名(形参1, 形参2, ...) { // 函数声明的小括号中的是形参
    // 函数体代码
}
函数名(实参1, 实参2, ...)           // 函数调用的小括号中的是实参
```



4.1 初识函数

4

函数参数的数量

函数的参数允许形参和实参的个数不同：

- ❑ 当实参数量多于形参数量时，函数正常执行，多余的实参会被忽略
- ❑ 当实参数量小于形参数量时，多出来的形参类似于一个已声明未赋值的变量，其值为undefined



4.1 初识函数

4

函数参数的数量

示例代码

```
function getSum(num1, num2) {  
  console.log(num1, num2);  
}  
  
getSum(1, 2, 3); // 实参数量大于形参数量, 输出结果: 1 2  
getSum(1);      // 实参数量小于形参数量, 输出结果: 1 undefined
```



4.1 初识函数

5

函数的返回值

函数的返回值可以将函数的处理结果返回，用于根据函数的执行结果来决定下一步要做的事情，函数的返回值通过return语句实现。

语法结构

```
function 函数名() {  
    return 要返回的值; // 利用return返回一个值给调用者  
}
```



4.1 初识函数

6

arguments的使用

当不确定函数中接收到了多少个实参的时候，可以用[arguments](#)来获取实参。这是因为**arguments**是当前函数的一个**内置对象**，所有函数都内置了一个arguments对象，该对象**保存了函数调用时传递的所有的实参**。



4.1 初识函数

6

arguments的使用

示例代码

```
function fn() {  
  console.log(arguments); // 输出结果: Arguments(3) [1, 2, 3, ...]  
  console.log(arguments.length); // 输出结果: 3  
  console.log(arguments[1]);    // 输出结果: 2  
}  
fn(1, 2, 3);
```



4.2 函数案例（自学）

1

利用函数求任意两个数的最大值

案例要求：编写一个getMax()函数，该函数接收两个参数，分别是num1和num2，表示两个数字。收到参数后，比较两个数的大小，返回较大的值。

示例代码

```
function getMax(num1, num2) {  
    return num1 > num2 ? num1 : num2;  
}  
console.log(getMax(1, 3)); // 输出结果： 3
```



4.2 函数案例

2

利用函数求任意一个数组中的最大值

案例要求: 利用函数求数组[5, 2, 99, 101, 67, 77]中的最大数值。

示例代码

```
function getArrMax(arr) {  
    var max = arr[0];  
    for (var i = 1; i <= arr.length; i++) {  
        if (arr[i] > max) { max = arr[i]; }  
    }  
    return max;  
}  
  
var arr = getArrMax([5, 2, 99, 101, 67, 77]);  
console.log(arr); // 输出结果: 101
```




4.2 函数案例

3

利用return提前终止函数

案例要求: 在getMax()函数中判断两个参数是否都是数字型，只要其中一个不是数字型，则提前返回NaN。

示例代码

```
function getMax(num1, num2) {  
  if (typeof num1 !== 'number' || typeof num2 !==  
    'number') {  
    return NaN;  
  }  
  return num1 > num2 ? num1 : num2;  
}  
console.log(getMax(1, '3')); // 输出结果: NaN
```



4.2 函数案例

4

利用return返回数组

案例要求: 在开发中, 当需要返回多个值的时候, 可以用数组来实现, 也就是将要返回的多个值写在数组中, 作为一个整体来返回。

示例代码

```
function getResult(num1, num2) {  
    return [num1 + num2, num1 - num2, num1 *  
    num2, num1 / num2];  
}  
  
console.log(getResult(1, 2));  
// 输出结果: (4) [3, -1, 2, 0.5]
```



4.2 函数案例

5

利用函数求所有参数中的最大值

案例要求: 编写一个getMax()函数，该函数可以传任意数量的参数，在函数中会找出所有参数中最大的一个值，将该值返回。

示例代码

```
function getMax() {  
    var max = arguments[0];  
    for (var i = 1; i < arguments.length; i++) {  
        if (arguments[i] > max) { max = arguments[i]; }  
    }  
    return max;  
}  
console.log(getMax(11, 2, 34, 666, 5, 100)); // 输出结果: 666
```



4.2 函数案例

6

利用函数反转数组元素顺序

案例要求: 编写一个reverse()函数，该函数的参数是一个数组，在函数中会对数组中的元素顺序进行反转，返回反转后的数组。

示例代码

```
function reverse(arr) {  
  var newArr = [];  
  for (var i = arr.length - 1; i >= 0; i--) {  
    newArr[newArr.length] = arr[i];  
  }  
  return newArr;  
}  
  
var arr1 = reverse([1, 3, 4, 6, 9]);  
console.log(arr1);           // 输出结果: (5) [9, 6, 4, 3, 1]
```



4.2 函数案例

7

利用函数判断闰年

案例要求: 编写一个isLeapYear()函数，该函数的参数是一个年份数字，在函数中会判断该年份是否为闰年，返回布尔值的结果。

示例代码

```
function isLeapYear(year) {  
  var flag = false;  
  if (year % 4 == 0 && year % 100 != 0 || year % 400 == 0) {  
    flag = true;  
  }  
  return flag;  
}  
  
console.log(isLeapYear(2020) ? '2020是闰年' : '2020不是闰年');  
console.log(isLeapYear(2021) ? '2021是闰年' : '2021不是闰年');
```



4.2 函数案例

8

获取指定年份的2月份的天数

案例要求: 编写一个fn()函数，该函数调用后会弹出一个输入框，要求用户输入一个年份，输入以后，程序会提示该年份的2月份有多少天。

示例代码

```
function fn() {  
    var year = prompt('请输入年份: ');  
    if (isLeapYear(year)) { alert( '当前年份是闰年, 2月份有29天' ); }  
    else { alert('当前年份是平年, 2月份有28天'); }  
}  
  
function isLeapYear(year) {  
    // 将上一个案例中的isLeapYear()函数中的代码复制到此处  
}  
  
fn();
```



4.3 函数进阶

1

函数表达式

函数表达式：是将声明的函数赋值给一个变量，通过变量完成函数的调用和参数的传递，**函数表达式的定义必须在调用前**。

匿名函数

语法结构

注意
前后
顺序

```
var sum = function(num1, num2) { // 函数表达式
    return num1 + num2;
};
console.log(sum(1, 2));           // 调用函数，输出结果：3
```



4.3 函数进阶

2

回调函数

回调函数：指的就是一个函数A作为参数传递给一个函数B，然后在B的函数体内调用函数A。此时，称函数A为回调函数。以算术运算为例进行讲解：

示例代码

```
function cal(num1, num2, fn) {  
    return fn(num1, num2);  
}  
  
console.log(cal(45, 55, function (a, b) {  
    return a + b;  
}));  
  
console.log(cal(10, 20, function (a, b) {  
    return a * b;  
}));
```




4.3 函数进阶

3

递归调用

递归调用：指的是一个函数在其函数体内调用自身的过程，这种函数称为递归函数。以根据用户的输入计算指定数据的阶乘为例进行讲解：

示例代码

```
function factorial(n) {    // 定义回调函数
    if (n == 1) { return 1; // 递归出 }
    return n * factorial(n - 1);
}
var n = prompt('求n的阶乘\n n是大于等于1的正整数，如2表示求2!。 ');
n = parseInt(n);
if (isNaN(n)) { console.log('输入的n值不合法') }
else { console.log(n + '的阶乘为： ' + factorial(n)); }
```



4.4 作用域

1

作用域的分类

变量需要在它的作用范围内才可以被使用，这个作用范围称为[变量的作用域](#)，

JavaScript根据作用域使用范围的不同，划分如下：

- ❑ **全局变量**：不在任何函数内声明的变量（显式定义）或在函数内省略**var**声明的变量（隐式定义）都称为全局变量
- ❑ **局部变量**：在函数体内利用**var**关键字定义的变量称为局部变量
- ❑ **块级变量**：ES6提供的**let**关键字声明的变量称为块级变量



4.4 作用域

2

全局变量和局部变量

两者的区别:

- ❑ 在全局作用域下，添加或省略var关键字都可以声明全局变量，全局变量在浏览器关闭页面的时候才会销毁，比较占用内存资源
- ❑ 在函数中，添加var关键字声明的变量是局部变量，省略var关键字时，如果变量在当前作用域下不存在，会自动向上级作用域查找变量。局部变量在函数执行完成后就会销毁，比较节约内存资源



4.4 作用域

2 全局变量和局部变量

```
// 全局作用域
var num = 10; // 全局变量
function fn() {
  // 局部作用域
  var num = 20; // 局部变量
  console.log(num); // 输出局部变量num的值，输出结果：20
}
fn();
console.log(num); // 输出全局变量10的值，输出结果：10
```

```
function fn() {
  num2 = 20;
}
fn();
console.log(num2); // 输出结果：
```

fn()查找局部作用域，无，
查找上级作用于，至全
局作用于，无，创建全
局变量num2



4.4 作用域

3

作用域链

作用域链：当在一个函数内部声明另一个函数时，内层函数只能在外层函数作用域内执行，在内层函数执行的过程中，若需要引入某个变量，首先会在当前作用域中寻找，若未找到，则继续向上一层级的作用域中寻找，直到全局作用域，称这种链式的查询关系为作用域链。

```
var num = 10;
function fn() { // 外部函数
    var num = 20;
    function fun() { // 内部函数
        console.log(num); // 输出结果：20
    }
    fun();
}
fn();
```

本章总结

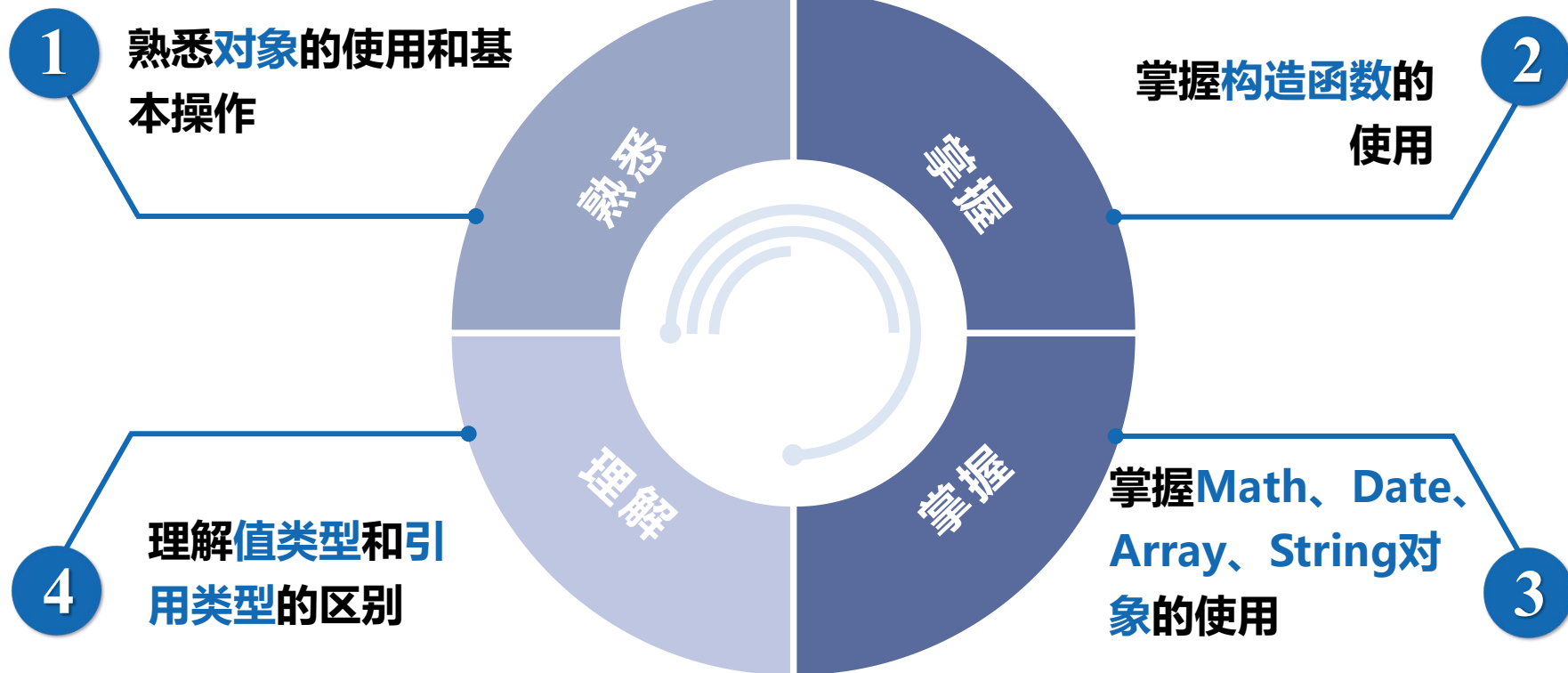


本章首先介绍了什么是函数、函数的使用、参数和返回值的设置，然后讲解了使用**arguments**来获取未知的实参个数及函数表达式，接着针对函数的全局变量和局部变量的作用域进行讲解。通过本章的学习，希望读者能够熟练掌握函数的使用。



第5章 JavaScript对象

学习目标



目录



5.1

初识对象

👉 [点击查看本节相关知识点](#)

5.2

内置对象

👉 [点击查看本节相关知识点](#)

5.3

Math对象

👉 [点击查看本节相关知识点](#)

5.4

日期对象

👉 [点击查看本节相关知识点](#)

目录



5.5

数组对象

👉 [点击查看本节相关知识点](#)

5.6

字符串对象

👉 [点击查看本节相关知识点](#)

5.7

值引用和引用类型

👉 [点击查看本节相关知识点](#)



5.1 初识对象

1

什么是对象

在JavaScript中，对象是一种数据类型，它是由属性和方法组成的一个集合。

属性是指事物的特征，使用“对象.属性名”访问；方法是指事物的行为，使用“对象.方法名()”进行访问。



5.1 初识对象

1

什么是对象

手机对象p1



属性

方法

```
var p1 = {  
  color: '黑色',  
  weight: '188g',  
  screenSize: '6.5',  
  call: function(num) { console.log('打电话给' + num); },  
  sendMessage: function(num, message) {},  
  playVideo: function() { console.log('播放视频'); },  
  playMusic: function() { console.log('播放音乐'); }  
}
```



5.1 初识对象

1

什么是对象

访问p1的属性
和方法

```
console.log(p1.color);  
console.log(p1.weight);  
console.log(p1.screenSize);  
p1.call('123');  
p1.sendMessage('123', 'hello');  
p1.playVideo();  
p1.playMusic();
```



5.1 初识对象

2 利用字面量创建对象

对象的字面量就是用花括号“{ }”来包裹对象中的成员，每个成员使用“**key: value**”的形式来保存，**key**表示属性名或方法名，**value**表示对应的值。多个对象成员之间用“**,**”隔开。



5.1 初识对象

2 利用字面量创建对象

创建一个空对象

```
var obj = { };
```

创建一个学
生对象

```
var stu1 = {  
  name: '小明',    // name属性  
  age: 18,         // age属性  
  sex: '男',       // sex属性  
  // sayHello方法  
  sayHello: function() { console.log('Hello'); } };
```



5.1 初识对象

3

访问对象的属性和方法

创建好学生对象之后，接下来访问该对象的属性和方法：

① 访问学生对象的属性：

语法结构

```
stu1.name    // 方式1，输出结果为：小明  
stu1['age']   // 方式2，输出结果为：18
```

② 访问学生对象的方法：

语法结构

```
stu1.sayHello()    // 方式1，输出结果为：Hello  
stu1['sayHello']() // 方式2，输出结果为：Hello
```




5.1 初识对象

3

访问对象的属性和方法

当对象成员中包含特殊字符时，可以用字符串来表示：

语法结构

```
var obj = {  
  'name-age': '小明-18'  
};  
console.log(obj['name-age']);    // 输出结果: “小明-18”
```



5.1 初识对象

3

访问对象的属性和方法

手动赋值属性或方法来**添加成员**:

示例代码

```
var stu2 = {};           // 创建一个空对象
stu2.name = 'Jack';      // 为对象增加name属性
stu2.introduce = function() { // 为对象增加introduce方法
    alert('My name is ' + this.name); // this代表当前对象
};
alert(stu2.name); // 访问name属性, 输出结果: Jack
```



5.1 初识对象

4

用new Object创建对象

示例代码

```
var obj = new Object();    // 创建了一个空对象
obj.name = '小明';        // 创建对象后，为对象添加成员
obj.age = 18;
obj.sex = '男';
obj.sayHello = function() {
  console.log('Hello');
};
```



5.1 初识对象

5

利用构造函数创建对象

使用构造函数创建对象的语法为“[new 构造函数名\(\)](#)”，在小括号中可以传递参数给构造函数，如果没有参数，小括号可以省略。



5.1 初识对象

5

利用构造函数创建对象

语法结构

```
// 编写构造函数
function 构造函数名() {
  this.属性 = 属性;
  this.方法 = function() {
    // 方法体
  }
}

// 使用构造函数创建对象
var obj = new 构造函数名();
```



13.3 this指向

1

this指向

函数中this指向，情况如下：

- ① 构造函数内部的**this**指向新创建的对象。
- ② 直接通过函数名调用函数时，**this**指向的是全局对象window。
- ③ 如果将函数作为对象的方法调用，**this**将会指向该对象。



5.1 初识对象

5

利用构造函数创建对象

```
// 编写一个Student构造函数
function Student(name, age) {
  this.name = name;
  this.age = age;
  this.sayHello = function () {
    console.log('你好, 我叫' + this.name);
  };
}
// 使用Student构造函数创建对象
var stu1 = new Student('小明', 18);
console.log(stu1.name); // 输出结果: 小明
console.log(stu1.sayHello()); // 输出结果: 你好, 我叫小明
var stu2 = new Student('小红', 17);
console.log(stu2.name); // 输出结果: 小红
console.log(stu2.sayHello()); // 输出结果: 你好, 我叫小红
```



5.1 初识对象

5

利用构造函数创建对象

```
// 静态static成员，构造函数本身就有属性和方法，不需要创建实例对象就能使用
function Student() {
}
Student.school = 'X大学';           // 添加静态属性school
Student.sayHello = function () {    // 添加静态方法sayHello
    console.log('Hello');
};
console.log(Student.school); // 输出结果: X大学
Student.sayHello();         // 输出结果: Hello
```




5.1 初识对象

6

遍历对象的属性和方法

使用[for...in语法](#)可以遍历对象中的所有属性和方法，示例代码如下：

示例代码

```
// obj为待遍历的对象
var obj = { name: '小明', age: 18, sex: '男' };
// 遍历obj对象
for (var k in obj) {
    // 通过k可以获取遍历过程中的属性名或方法名
    console.log(k);           // 依次输出： name、 age、
sex
    console.log(obj[k]);      // 依次输出： 小明、 18、 男
}
```



5.1 初识对象

6

遍历对象的属性和方法

多学一招

使用[in运算符](#)判断一个对象中的某个成员是否存在。

```
var obj = {name: 'Tom', age: 16};  
console.log('age' in obj);    // 输出: true, 表示对象成员存在  
console.log('gender' in obj); // 输出: false, 表示对象成员不存在
```



5.2 内置对象

1

通过查阅文档熟悉内置对象

JavaScript提供了很多常用的内置对象，包括数学对象Math、日期对象Date、数组对象Array以及字符串对象String等。我们以Mozilla开发者网络（MDN）为例，演示如何查阅JavaScript中的内置对象的使用。

<https://developer.mozilla.org/zh-CN/docs/Web>



5.2 内置对象

1

通过查阅文档熟悉内置对象

① 打开MDN网站，在网站的导航栏中找到“技术” - “JavaScript”，效果如下。



MDN文档



5.2 内置对象

1

通过查阅文档熟悉内置对象

- ② 将页面向下滚动，在左侧边栏中找到“内置对象”，将该项展开后，可以看到所有内置对象的目录链接，如下图所示。



内置对象目录

5.2 内置对象

1

通过查阅文档熟悉内置对象

- ③ 如果不知道对象的名称，也可以在页面的搜索框中输入关键字进行搜索、查找。以`Math`为例，效果图如下所示。



使用说明



5.2 内置对象

1

通过查阅文档熟悉内置对象

④ 通过文档学习某个方法的使用时，基本上可以分为4个步骤：

- ❑ 查阅方法的功能
- ❑ 查看参数的意义和类型
- ❑ 查看返回值的意义和类型
- ❑ 通过示例代码进行测试



5.2 内置对象

2

【案例】封装自己的数学对象

```
console.log(myMath.PI);  
// 输出结果: 3.141592653589793  
console.log(myMath.max(10, 20, 30));  
// 输出结果: 30
```

案例要求: 封装一个数学对象myMath, 实现求出数组中的最大值。

示例代码

```
var myMath = {  
  PI: 3.141592653589793,  
  max: function() {  
    var max = arguments[0];  
    for (var i = 1; i < arguments.length; i++) {  
      if (arguments[i] > max) { max = arguments[i]; }  
    }  
    return max;  
  }  
};
```




5.3 Math对象

1

Math对象的使用

Math对象用来对数字进行与数学相关的运算，不需要实例化对象，可以直接使用其静态属性和静态方法，**Math**对象的常用属性和方法如下表。

成员	功能
PI	获取圆周率，结果为3.141592653589793
abs(x)	获取x的绝对值，可传入普通数值或是用字符串表示的数值
max()	获取所有参数中的最大值
min()	获取所有参数中的最小值
pow(base, exponent)	获取基数（base）的指数（exponent）次幂，即 base ^{exponent}
sqrt(x)	获取x的平方根
ceil(x)	获取大于或等于x的最小整数，即向上取整
floor(x)	获取小于或等于x的最大整数，即向下取整
round(x)	获取x的四舍五入后的整数值
random()	获取大于或等于0.0且小于1.0的随机值



5.3 Math对象

2

生成指定范围的随机数

[Math.random\(\)](#)用来获取随机数，每次调用该方法返回的结果都不同。该方法返回的结果是一个很长的浮点数，其范围是0~1（不包括1）。

示例代码

```
// 表示生成大于或等于min且小于max的随机值  
Math.random() * (max - min) + min;  
// 表示生成0到任意数之间的随机整数  
Math.floor(Math.random() * (max + 1));  
// 表示生成1到任意数之间的随机整数  
Math.floor(Math.random() * (max + 1) + 1);
```



5.3 Math对象

3

【案例】猜数字游戏

案例需求: 使程序随机生成一个1~10之间的数字，并让用户输入一个数字，判断这两个数的大小，如果用户输入的数字大于随机数，那么提示“你猜大了”，如果用户输入的数字小于随机数，则提示“你猜小了”，如果两个数字相等，就提示“恭喜你，猜对了”，结束程序。



5.3 Math对象

3

【案例】猜数字游戏

示例代码

```
function getRandom(min, max) {  
    return Math.floor(Math.random() * (max - min + 1) + min);  
}  
var random = getRandom(1, 10);  
while (true) {  
    var num = prompt('猜数字， 范围在1~10之间。');  
    if (num > random) { alert('你猜大了'); }  
    else if (num < random) { alert('你猜小了'); }  
    else { alert('恭喜你， 猜对了'); break; }  
}
```



5.4 日期对象

1

日期对象的使用

JavaScript中的[日期对象](#)需要使用`new Date()`实例化对象才能使用，`Date()`是日期对象的构造函数。`Date()`构造函数可以传入一些参数，示例代码如下。



5.4 日期对象

1

日期对象的使用

示例代码

```
// 方式1: 没有参数
var date1 = new Date();
// 输出: Wed Oct 16 2019 10:57:56 GMT+0800 (中国标准时间)
// 方式2: 传入年、月、日、时、分、秒 (月的范围是0~11)
var date2 = new Date(2019, 10, 16, 10, 57, 56);
// 输出: Sat Nov 16 2019 10:57:56 GMT+0800 (中国标准时间)
// 方式3: 用字符串表示日期和时间
var date3 = new Date('2019-10-16 10:57:56');
// 输出: Wed Oct 16 2019 10:57:56 GMT+0800 (中国标准时间)
```



5.4 日期对象

1

日期对象的使用

日期对象的常用get方法

方法	作用
getFullYear()	获取表示年份的4位数字，如2020
getMonth()	获取月份，范围0~11（0表示一月，1表示二月，依次类推）
getDate()	获取月份中的某一天，范围1~31
getDay()	获取星期，范围0~6（0表示星期日，1表示星期一，依次类推）
getHours()	获取小时数，返回0~23
getMinutes()	获取分钟数，范围0~59
getSeconds()	获取秒数，范围0~59
getMilliseconds()	获取毫秒数，范围0~999
getTime()	获取从1970-01-01 00:00:00距离Date对象所代表时间的毫秒数



5.4 日期对象

1

日期对象的使用

日期对象的常用set方法

方法	作用
setFullYear(value)	设置年份
setMonth(value)	设置月份
setDate(value)	设置月份中的某一天
setHours(value)	设置小时数
setMinutes(value)	设置分钟数
setSeconds(value)	设置秒数
setMilliseconds(value)	设置毫秒数
setTime(value)	通过从1970-01-01 00:00:00计时的毫秒数来设置时间



5.4 日期对象

2

【案例】统计代码执行时间

时间戳是获取从1970年1月1日0时0分0秒开始一直到当前UTC时间所经过的毫秒数。

获取时间戳的常见方式如下：

```
// 方式1：通过日期对象的valueOf()或getTime()方法
var date1 = new Date();
console.log(date1.valueOf()); // 示例结果：1571196996188
console.log(date1.getTime()); // 示例结果：1571196996188
// 方式2：使用 “+” 运算符转换为数值型
var date2 = +new Date();
console.log(date2);           // 示例结果：1571196996190
// 方式3：使用HTML5新增的Date.now()方法
console.log(Date.now());     // 示例结果：1571196996190
```



5.4 日期对象

3

【案例】倒计时

案例需求：在一些电商网站的活动页上会经常出现折扣商品的倒计时标记，显示离活动结束还剩X天X小时X分X秒。

倒计时的核心算法是输入的时间减去现在的时间，得出的剩余时间就是要显示的倒计时时间，这要把时间都转化成时间戳（毫秒数）来进行计算，把得到的毫秒数转换为天数、小时、分数、秒数。



5.4 日期对象

3

【案例】倒计时

示例代码

```
function countdown(time) {  
    var nowTime = +new Date();  
    var inputTime = +new Date(time);  
    var times = (inputTime - nowTime) / 1000;  
    var d = parseInt(times / 60 / 60 / 24); d = d < 10 ? '0' + d : d;  
    var h = parseInt(times / 60 / 60 % 24); h = h < 10 ? '0' + h : h;  
    var m = parseInt(times / 60 % 60); m = m < 10 ? '0' + m : m;  
    var s = parseInt(times % 60); s = s < 10 ? '0' + s : s;  
    return d + '天' + h + '时' + m + '分' + s + '秒';  
}  
  
// 示例结果: 05天23时06分10秒  
console.log(countdown('2019-10-22 10:56:57'));
```



5.5 数组对象

例如对传入参数的检测

1

数组类型检测

数组类型检测有两种常用的方式，分别是使用[instanceof运算符](#)和使用

[Array.isArray\(\)方法](#)。

示例代码

```
var arr = [];  
var obj = {};  
// 第1种方式  
console.log(arr instanceof Array); // 输出结果: true  
console.log(obj instanceof Array); // 输出结果: false  
// 第2种方式  
console.log(Array.isArray(arr)); // 输出结果: true  
console.log(Array.isArray(obj)); // 输出结果: false
```



5.5 数组对象

2 添加或删除数组元素

JavaScript数组对象提供了添加或删除元素的方法，可以实现在数组的末尾或开头添加新的数组元素，或在数组的末尾或开头移出数组元素。方法如下：

方法名	功能描述	返回值
push(参数1...)	数组末尾添加一个或多个元素，会修改原数组	返回数组的新长度
unshift(参数1...)	数组开头添加一个或多个元素，会修改原数组	返回数组的新长度
pop()	删除数组的最后一个元素，若是空数组则返回undefined，会修改原数组	返回删除的元素的值
shift()	删除数组的第一个元素，若是空数组则返回undefined，会修改原数组	返回第一个元素的值



5.5 数组对象

2 添加或删除数组元素

注意

`push()`和`unshift()`方法的返回值是新数组的长度，而`pop()`和`shift()`方法返回的是移出的数组元素。



5.5 数组对象

3

【案例】筛选数组

案例需求：要求在包含工资的数组中，剔除工资达到2000或以上的数据，把小于2000的数重新放到新的数组里面。

示例代码

```
var arr = [1500, 1200, 2000, 2100, 1800];
var newArr = [];
for (var i = 0; i < arr.length; i++) {
  if (arr[i] < 2000) {
    newArr.push(arr[i]); // 相当于: newArr[newArr.length] = arr[i];
  }
}
console.log(newArr); // 输出结果: (3) [1500, 1200, 1800]
```



5.5 数组对象

4

数组排序

JavaScript数组对象提供了[数组排序](#)的方法，可以实现数组元素排序或者颠倒数组元素的顺序等。排序方法如下：

方法	功能描述
reverse()	颠倒数组中元素的位置，该方法会改变原数组，返回新数组
sort()	对数组的元素进行排序，该方法会改变原数组，返回新数组



5.5 数组对象

4

数组排序

注意

需要注意的是：`reverse()`和`sort()`方法的返回值是新数组的长度。



5.5 数组对象

5

数组索引

在开发中，若要查找指定的元素在数组中的位置，可以利用**Array**对象提供的检索方法。检索方法如下：

方法	功能描述
indexOf()	返回在数组中可以找到给定值的第一个索引，如果不存在，则返回-1
lastIndexOf()	返回指定元素在数组中的最后一个的索引，如果不存在，则返回-1



5.5 数组对象

5

数组索引

注意

默认都是从指定数组索引的位置开始检索，并且检索方式与运算符“===”相同，即只有全等时才会返回比较成功的结果。



5.5 数组对象

6

【案例】数组去除重复元素

案例需求: 要求在一组数据中，去除重复的元素。

示例代码

```
function unique(arr) {  
  var newArr = [];  
  for (var i = 0; i < arr.length; i++) {  
    if (newArr.indexOf(arr[i]) === -1) { newArr.push(arr[i]); }  
  }  
  return newArr;  
}  
  
var demo = unique(['blue', 'green', 'blue']);  
console.log(demo);           // 输出结果: (4) ["blue", "green"]
```



5.5 数组对象

7

数组转换为字符串

在开发中，可以利用数组对象的`join()`和`toString()`方法，将数组转换为字符串。

方法如下：

方法	功能描述
<code>toString()</code>	把数组转换为字符串，逗号分隔每一项
<code>join('分隔符')</code>	将数组的所有元素连接到一个字符串中



5.5 数组对象

7

数组转换为字符串

示例代码

```
// 使用toString()
var arr = ['a', 'b', 'c'];
console.log(arr.toString()); // 输出结果: a,b,c
// 使用join()
console.log(arr.join());      // 输出结果: a,b,c
console.log(arr.join(''));    // 输出结果: abc
console.log(arr.join('-'));    // 输出结果: a-b-c
```



5.5 数组对象

8

其他方法

JavaScript还提供了很多其他常用的数组方法。例如，填充数组、连接数组、截取数组元素等。方法如下：

方法	功能描述
fill()	用一个固定值填充数组中指定下标范围内的全部元素
splice()	数组删除，参数为splice(第几个开始, 要删除个数)，返回被删除项目的新数组
slice()	数组截取，参数为slice(begin, end)，返回被截取项目的新数组
concat()	连接两个或多个数组，不影响原数组，返回一个新数组



5.5 数组对象

8

其他方法

注意

`slice()`和`concat()`方法在执行后返回一个新的数组，不会对原数组产生影响，剩余的方法在执行后皆会对原数组产生影响。



5.6 字符串对象

1

字符串对象的使用

字符串对象使用 `new String()` 来创建，在 `String` 构造函数中传入字符串，这样就会在返回的字符串对象中保存这个字符串。

示例代码

```
var str = new String('apple');    // 创建字符串对象
console.log(str);                 // 输出结果: String {"apple"}
console.log(str.length);          // 获取字符串长度, 输出结果: 5
```



5.6 字符串对象

2

根据字符返回位置

字符串对象提供了用于检索元素的属性和方法，字符串对象的常用属性和方法如下：

方法	功能描述
indexOf(searchValue)	获取searchValue在字符串中首次出现的位置
lastIndexOf(searchValue)	获取searchValue在字符串中最后出现的位置



5.6 字符串对象

2

根据字符返回位置

示例代码

```
var str = 'HelloWorld';  
str.indexOf('o');    // 获取 “o” 在字符串中首次出现的位置，返回结果：4  
str.lastIndexOf('o'); // 获取 “o” 在字符串中最后出现的位置，返回结果：6
```



5.6 字符串对象

2

根据字符返回位置

案例需求: 要求在一组字符串中, 找到所有指定元素出现的位置以及次数。字符串为 'Hello World, Hello JavaScript'。

示例代码

```
var str = 'Hello World, Hello JavaScript';
var index = str.indexOf('o');
var num = 0;
while (index != -1) {
    console.log(index);           // 依次输出: 4、7、17
    index = str.indexOf('o', index + 1);
    num++;
}
console.log('o出现的次数是: ' + num); // o出现的次数是: 3
```



5.6 字符串对象

3

根据位置返回字符

字符串对象提供了用于获取字符串中的某一个字符的方法。方法如下：

成员	作用
<code>charAt(index)</code>	获取index位置的字符，位置从0开始计算
<code>charCodeAt(index)</code>	获取index位置的字符的ASCII码
<code>str[index]</code>	获取指定位置处的字符（HTML5新增）



5.6 字符串对象

3

根据位置返回字符

示例代码

```
var str = 'HelloWorld';  
var str = 'Apple';  
console.log(str.charAt(3));      // 输出结果： 1  
console.log(str.charCodeAt(0));  // 输出结果： 65 (字符A的ASCII码为65)  
console.log(str[0]);             // 输出结果： A
```



5.6 字符串对象

4

【案例】统计出现最多的字符和次数

案例需求: 使用charAt()方法通过程序来统计字符串中出现最多的字符和次数。

示例代码

```
var str = 'Apple';  
// 第1步, 统计每个字符的出现次数  
var o = {};  
for (var i = 0; i < str.length; i++) {  
    var chars = str.charAt(i); // 利用chars保存字符串中的每一个字符  
    if (o[chars]) {           // 利用对象的属性来方便查找元素  
        o[chars]++;  
    } else { o[chars] = 1; }  
}  
console.log(o);               // 输出结果: {A: 1, p: 2, l: 1, e: 1}
```



5.6 字符串对象

4

【案例】统计出现最多的字符和次数

示例代码

```
// 第2步，统计出现最多的字符
var max = 0;           // 保存出现次数最大值
var ch = '';           // 保存出现次数最多的字符
for (var k in o) {
  if (o[k] > max) {
    max = o[k];
    ch = k;
  }
}
// 输出结果：“出现最多的字符是：p，共出现了2次”
console.log('出现最多的字符是：' + ch + '，共出现了' + max + '次');
```




5.6 字符串对象

5

字符串操作方法

字符串对象提供了一些用于截取字符串、连接字符串、替换字符串的属性和方法。

字符串对象的常用属性和方法如下：

方法	作用
<code>concat(str1, str2, str3...)</code>	连接多个字符串
<code>slice(start,[end])</code>	截取从start位置到end位置之间的一个子字符串
<code>substring(start[, end])</code>	截取从start位置到end位置之间的一个子字符串，基本和slice相同，但是不接收负值
<code>substr(start[, length])</code>	截取从start位置开始到length长度的子字符串
<code>toLowerCase()</code>	获取字符串的小写形式
<code>toUpperCase()</code>	获取字符串的大写形式
<code>split([separator[, limit])</code>	使用separator分隔符将字符串分隔成数组，limit用于限制数量
<code>replace(str1, str2)</code>	使用str2替换字符串中的str1，返回替换结果，只会替换第一个字符



5.6 字符串对象

5

字符串操作方法

示例代码

```
var str = 'HelloWorld';  
str.concat('!'); // 在字符串末尾拼接字符, 结果: HelloWorld!  
str.slice(1, 3); // 截取从位置1开始包括到位置3的范围内的内容, 结果: el  
str.substring(5); // 截取从位置5开始到最后的内容, 结果: World  
str.substring(5, 7); // 截取从位置5开始到位置7范围内的内容, 结果: Wo  
str.substr(5); // 截取从位置5开始到字符串结尾的内容, 结果: World  
str.substring(5, 7); // 截取从位置5开始到位置7范围内的内容, 结果: Wo  
str.toLowerCase(); // 将字符串转换为小写, 结果: helloworld  
str.toUpperCase(); // 将字符串转换为大写, 结果: HELLOWORLD  
str.split('l'); // 使用 "l" 切割字符串, 结果: ["He", "", "oWor", "d"]  
str.split('l', 3); // 限制最多切割3次, 结果: ["He", "", "oWor"]  
str.replace('World', '!'); // 替换字符串, 结果: "Hello!"
```



5.6 字符串对象

6

【案例】判断用户名是否合法

案例需求: 用户名长度在3~10范围内，不能出现敏感词admin的任何大小写形式。

示例代码

```
var name = prompt('请输入用户名');  
if (name.length < 3 || name.length > 10) {  
    alert('用户名长度必须在3~10之间。');  
} else if (name.toLowerCase().indexOf('admin') !== -1) {  
    alert('用户名中不能包含敏感词: admin。');  
} else {  
    alert('恭喜您, 该用户名可以使用');  
}
```

本章总结



本章首先讲解了对对象的基本概念，然后讲解了如何自定义对象、如何使用内置对象，并通过使用日期对象实现倒计时功能，通过数组对象实现数组排序、根据索引检索元素，以及去除数组中的重复元素，使用字符串对象提供的方法实现字符串截取、替换等。通过本章学习，读者应能够通过内置对象完成实际开发需求。